

[Small Font Size Demo](#)

[This document demonstrates very small font size of 8pt.](#)

[Small fonts are used in footnotes and disclaimers.](#)

# eigenpal/docx-editor

Project Charter & Contributor Agreement

npm [@eigenpal/docx-js-editor](#)

github [eigenpal/docx-editor](#)

## 1. What this is

An open-source WYSIWYG .docx editor for React. Drop in `<DocxEditor documentBuffer={file} />` and your users can open, edit, and save Word documents entirely in the browser — no backend required —, no document data leaving the client.

Most editors serialise to HTML and back: `DOCX → HTML → edits → HTML → DOCX`. Every hop loses formatting, tracked changes, and styles. This library skips it — OOXML is the canonical format throughout.

```
import { DocxEditor } from '@eigenpal/docx-js-editor'
import '@eigenpal/docx-js-editor/styles.css'

function App({ file }: { file: ArrayBuffer }) {
  return (
    <DocxEditor
      documentBuffer={file}
      onSave={({buffer}) => uploadToServer(buffer)}
    />
  )
}
```

## 2. Collaboration — Tracked Changes & Comments

Both features round-trip through OOXML so documents open correctly in Word, LibreOffice, and Google Docs.

### Suggestion mode

Toggle between Editing and Suggesting via the toolbar dropdown. In Suggesting mode, every edit becomes a tracked insertion ( `green underline` ) or deletion ( `red strikethrough` ). Consecutive keystrokes group into a single change. Accept or reject individually or all at once.

Serialises to `<w:ins />` the same OOXML markup Word uses. Tracked changes survive a save/reload cycle and are reviewable in Word with full accept/reject UI.

Comments

Select text and click the floating button on the page edge to add a comment anchored to that selection. A Google Docs-style sidebar shows threaded cards positioned at their anchor point in the document. Reply, resolve, reopen, or delete inline.

Serialises to `<comments.xml>` with `<w:commentRangeStart />` `<w:commentRangeEnd>` markers. Comments and reply threads are preserved on round-trip.

Agent-friendly

Because tracked changes and comments are native OOXML, an AI agent can write `<w:ins />` `<w:del />` `<w:comment>` markup directly into a document and hand it to `<DocxEditor />` for human review — no custom diff format, no proprietary API.

3. Props

Pass `author="Your App"` to set the author name for comments and tracked changes. Toggle suggestion mode via the toolbar dropdown.

| Prop           | Type        | Default | Description                                  |
|----------------|-------------|---------|----------------------------------------------|
| documentBuffer | ArrayBuffer | —       | .doex file contents to load                  |
| readOnly       | boolean     | false   | Preview mode — no editing UI                 |
| showToolbar    | boolean     | true    | Show formatting toolbar                      |
| author         | string      | 'User'  | Author name for comments and tracked changes |
| onChange       | function    | —       | Called on every document change              |
| onSave         | function    | —       | Called on save, receives ArrayBuffer         |
| onError        | function    | —       | Called on error                              |

## Ref methods

```
const ref = useRef<DocxEditorRef>(null)

await ref.current.save() // ArrayBuffer of the .docx
ref.current.getDocument() // current document object
ref.current.setZoom(1.5) // set zoom to 150%
ref.current.scrollToPage(3) // scroll to page 3
ref.current.print() // print
```

## 4. Contributing

Read [docs/PLUGINS.md](#) before opening a pull request that touches the plugin API.

### Ground rules

- Every PR must pass the round-trip test: load a .docx, apply an edit, export, reload — content and formatting must be stable on unchanged content.
- Run `bun test` and `bun run test:e2e` locally before pushing.
- Avoid intermediate HTML serialisation where possible.
- Tracked changes and comments must survive a full OOXML round-trip — save, open in Word, verify changes are reviewable, reload in the editor.

### Dev setup

```
bun install
bun run dev // vite demo on localhost:5173
```

# eigenpal/docx-editor

## Project Charter & Contributor Agreement

npm [@eigenpal/docx-js-editor](#)

github [eigenpal/docx-editor](#)

## 1. What this is

An open-source WYSIWYG `.docx` editor for React. Drop in `<DocxEditor documentBuffer={file} />` and your users can open, edit, and save Word documents entirely in the browser — no backend required —, no document data leaving the client.

Most editors serialise to HTML and back: `DOCX → HTML → edits → HTML → DOCX`. Every hop loses formatting, tracked changes, and styles. This library skips it — OOXML is the canonical format throughout.